

> kdino

init:

Bine ați venit în sistemul de operare, DinoOS!
Un sistem de operare al informaticienilor pentru informaticieni, utilizatorii nu sunt bineveniți!

cpu:

DinoOS este executat pe un procesor pe 16-biți

mem:

Sistemul are memoria maximă adresabilă de procesor cu paginare 4K.

disk:

Sistemul are memorie secundară de 16 ori mai mare decât memoria principală
cu blocuri de dimensiunea unei pagini.

fs:

Disk-ul este formatat cu un sistem de fișier cu indexare limitat la un singur nivel de indexare.

1. (5p) Bifează răspunsurile corecte din oficiu. Nu vreau să fiu cetățeanul Planetei Utilizatorilor pentru că:

- ☐ ajung să cumpăr aplicații ce devin depășite înainte să apuc să le folosesc
☐ voi suferi de lipsă acută de atenție folosind aplicații gratuite în care de fapt eu sunt produsul
☐ mă voi bucura când un robot îmi va face tema, îmi va face munca, îmi va trăi viața
☐ voi ajunge să nu am nimic al meu și să fiu fericit

2. (20p) Memorie.

(a) (2p) Care este dimensiunea memoriei principale?

(b) (2p) Cum este implementată paginarea pe DinoOS? Câte frame-uri sunt în total?

(c) (2p) Reprezentați grafic structura memoriei.

(d) (10p) Fie un proces cu tabela de pagini din dreapta (index pagini lângă tabelă, index frame-uri în tabelă). Traduceți următoarele adrese logice în adrese fizice știind că un frame are dimensiunea unei pagini: 512, 4100, 8191, 8200, 16368. Scrieți adresele fizice în binar.

0	3
1	0
2	1
3	2

(e) (4p) Dacă se execută simultan programele de la punctul 3f), încap acestea complet în memorie? Arătați grafic cum se umple memoria.

3. (20p) În sistemul de fișiere numele directoarelor au maxim 16 caractere și numele fișierelor maxim 11 caractere.

O entitate de tip director este structurată astfel: la început avem numele directorului iar apoi avem câte o intrare pentru fiecare entitate (de tip fișier sau director) ce conțin informații legate de nume împreună cu legătura către blocul de start al fișierului sau directorului.

O entitate de tip fișier este alcătuită din blocul de indexare (și blocurile cu date).

(a) (2p) Câte blocuri are disk-ul? Care este dimensiunea totală a disk-ului?

(b) (2p) Sistemul de fișiere poate accesa toate blocurile. Descrieți structura unui bloc de indexare.

(c) (2p) Care este dimensiunea maximă a unui fișier?

(d) (2p) Care este numărul maxim de fișiere pe care le poate stoca sistemul de fișiere?

(e) (2p) Dacă tabela de tip director este ținută pe un singur bloc, câte fișiere poate conține maxim un director?

(f) (10p) Sistemul de fișiere conține directoarele rădăcină, /bin și /home împreună cu fișierele /kdino, /bin/sh și /bin/ed de dimensiune 32KB, 10KB și, respectiv, 3KB. Scrieți reprezentarea pe blocuri a acestui sistem de fișiere cu tot cu legăturile între blocuri.

p) Informaticianul DinoOS pornește sistemul kdino după care se autentifică în shell-ul sh și pornește editorul ed unde implementează și execută următorul pseudo-cod.

```

1 fork()
2 if (fork())
3     if (fork())
4         fork()

```

- (a) (5p) Câte procese și fire de execuție sunt la final? Desenați arborarea de procese și fire de execuție aferente.
- (b) (5p) Dacă avem memoria virtuală cu paginată la nivel și copy-on-write, descrieți grafic cum arată memoria principală după execuția de la punctul precedent.
- (c) (5p) Fie următoarea secvență de procese în lista de așteptare, unde **Priority** este prioritatea

Proces	Arrival	Burst	Priority
forky0	20	6	1
sh	16	4	1
forky1	20	5	4
ed	17	2	1
forky2	20	7	4
kdino	0	16	0
forky3	20	4	4

proceselor (numerele mici au prioritate mai mare). Cum arată diagrama Gantt rezultată în urma aplicării algoritmului SJF preemptiv ce ia în considerare întâi prioritatea și pe urmă timpul de execuție?

- (d) (5p) Dar folosind RR care ordonează apriori după prioritate cu cuantă $q = 4$?
- (e) (5p) Calculați timpul mediu de așteptare de la punctele anterioare.
5. (5p) DinoOS are nevoie de mutex pentru forky. Aveți la dispoziție pe procesor instrucțiunea TAS. Scrieți doar secvențele de pseudo-cod pentru funcțiile de lock/unlock.
6. (10p) Kernelul kdino cedează din numărul de frame-uri pentru a fi folosite de alte procese rămânând cu 4 frame-uri.
- (a) (2p) La câte frame-uri a renunțat știind că înainte nu avea nici un page-fault după încărcarea în memorie?
- (b) (8p) Acum are următoarea coadă de așteptare a paginilor: 0, 2, 1, 3, 4, 0, 4, 1, 2, 2, 6, 5, 2. Care este numărul optim de page-fault-uri? Care este numărul obținut de LRU? Ilustrați cum arată aplicarea algoritmilor în ambele cazuri.

wq

halt -p